Personal Finance Agent

Detailed use case and implementation roadmap

Build a financial wellness assistant that turns uploaded transaction data into explainable spending insights, budget monitoring, unusual-activity alerts and cash-flow forecasts. This specification translates the supplied PS-05 brief into concrete development tasks and a reproducible local demonstration.

Document status: Proposed implementation specification, version 1.0, 12 October 2026. The original PS-05 document supplies the product intent. The architecture, priorities, formulas, API contracts, datasets and acceptance targets below are added design recommendations; they are not organizer-issued requirements or a claim that software has already been built.

The Reg IRIS use case is used as a structural reference: background, objectives, architecture, definition of done, technology choices, tasks, inputs, guardrails and engineering expectations. Its regulatory rules and infrastructure are not transferred into this finance product.

How to use this specification

Read pages 2-6 to understand the product and system. Freeze pages 7-15 as the shared data and API contract before connecting the frontend. Use pages 16-19 for deployment and coding. Use pages 20-24 to prepare fixtures, tests and the final demonstration.

Pages	Contents
2-3	Problem, objectives, scope and measurable definition of done
4-6	High-level architecture, stack, services and agent responsibilities
7-8	Ingestion pipeline, transaction schema and database model
9-12	Budgeting, scoring, anomalies, forecasting and advice
13-15	Screens, REST endpoints, configuration and job contracts
16-19	Docker operation, Compose template and repository roadmap
20-22	Detailed tasks, dataset design and verification
23-24	Guardrails, failure handling, demo and references

Baseline assumptions: single-user-at-a-time local demo, synthetic INR data, manual statement uploads, CPU execution and no external API key requirement. Ownership checks still isolate two seeded users. Live banking, payment execution and investment trading remain outside the prototype.

Problem background and objective

I Use case short title

PS-05 Personal Finance Agent - Financial Wellness Monitoring and Planning

II Problem statement and background

A person may receive salary in one account, pay through UPI, use a credit card and hold subscriptions across several merchants. Statements show transactions, but they do not automatically explain rising expenses, approaching bills or the effect of a spending decision on next month’s balance. Reviewing separate files and maintaining budgets manually makes early intervention difficult.

The target user is a salaried individual who uploads three complete months of history and enters a current balance, recurring obligations and financial goals. The user needs an assistant that identifies a problem, explains its evidence, proposes an action and checks the outcome after the next upload.

III Objective

Implement a repeatable loop: ingest and validate transactions; classify spending; compute financial indicators; detect risks; forecast available cash; generate an evidence-backed action; and monitor again when new data or a scheduled run arrives. Every numerical conclusion must come from a deterministic calculation that can be inspected.

Functional domain	Responsibility and value
Financial Data Domain	Upload, parsing, normalization, source lineage and category review. Establishes trustworthy financial records.
Financial Intelligence Domain	Spending trends, budgets, anomalies, recurring bills, health indicators, forecasts and goals. Calculates what changed and why.
Advisory and Monitoring Domain	Orchestrates analysis, ranks insights, proposes actions and manages in-app alerts. Makes the findings useful to the user.

Expected outcomes

• A transaction can be traced to its statement page or CSV row and corrected without editing the raw source.

• A budget or anomaly alert exposes the transactions, period, threshold and calculation behind it.

• A forecast lists assumptions and upcoming obligations, giving the user a chance to act before an expected shortfall.

Scope boundaries

The baseline supports a declared CSV schema and one documented layout of a text-based PDF statement. “Always on” means the local scheduler rechecks stored data while the application is running. It does not mean live bank access or fresh data when no new statement has been uploaded.

Definition of done and priorities

The following are proposed acceptance requirements for this implementation. Must-have items define a complete baseline. Good-to-have items are added after the baseline is demonstrated.

ID	Tier	Observable acceptance condition
D01	Must	CSV plus one supported text-PDF fixture import successfully; invalid rows and unsupported PDFs are reported, not silently discarded.
D02	Must	Reviewable ledger includes category, account, direction, amount and a source link. User corrections persist and take precedence.
D03	Must	Dashboard shows month totals, spending trend, budget utilization, alerts and upcoming obligations from stored data.
D04	Must	Anomaly and recurring-charge detection return evidence and configurable thresholds; possible duplicates are reviewed by the user.
D05	Must	30-day cash-flow forecast shows daily balance, opening-balance date, obligations, assumptions and any shortfall date.
D06	Must	Health indicators and at least one saving action show inputs and arithmetic; missing inputs produce unavailable components.
D07	Must	Upload and category or budget changes trigger recomputation. Scheduler updates due-bill and stale-data alerts.
D08	Must	All baseline runtime services and initialization launch through docker compose up --build -d after documented .env setup.
D09	Must	OpenAPI documents routes and errors. Jobs survive process restarts without duplicate records or alerts.
D10	Must	At least 12 meaningful automated tests and one full upload-to-alert scenario pass in CI; restart preserves data.
D11	Good	Conversational advisor calls authorized tools and cites computed insight IDs; baseline works when the model is unavailable.
D12	Good	Goal planning, a scenario simulator and optional family view have explicit inputs and isolated permissions.

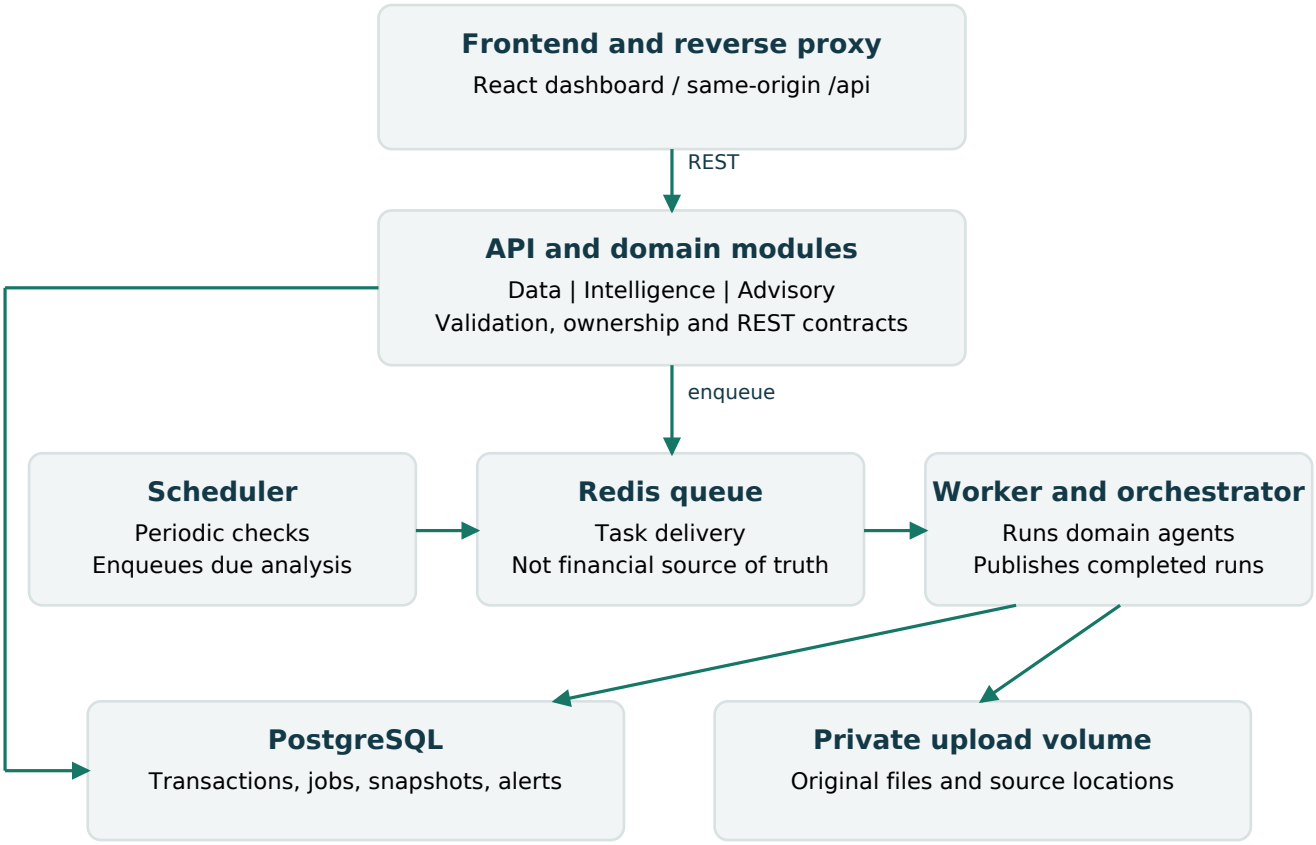
What the baseline must visibly demonstrate

One upload creates a normalized ledger, a budget warning, a recurring-charge candidate and a forecast. The user opens the evidence, changes a category or budget, and sees an updated result. The same upload repeated must not increase the totals.

Design note. Targets and minimum test counts are design proposals. No organizer deadline, judging weights or supplied dataset size is assumed.

High level architecture

Use one modular backend codebase with three domain boundaries, deployed as an API process, a worker process and a scheduler process. Ten logical agents are modules with explicit inputs and outputs; they do not require ten separate servers.



Data flow

The API stores a private upload and a durable job record. The worker parses it, persists reviewed or valid transactions, then computes a versioned analysis snapshot and insights. The frontend polls job status and renders only a completed snapshot. A scheduled run reuses stored data and may flag that the source is stale.

Deployment boundary

PostgreSQL is the durable source of truth. Redis transports tasks. API, worker and scheduler share the same application image and contracts. A one-shot initialization container applies migrations and optionally seeds synthetic data before runtime services start. A model service is optional.

Design note. This baseline is a modular multi-process application, not independently deployed domain microservices. If required later, extract the three domains behind REST or events; preserve their schemas and avoid cross-service table writes.

Technology stack and runtime services

This stack is a recommended baseline, chosen for a small team and a clear demo. Pin compatible dependency versions in lockfiles after the first smoke test; do not use floating latest tags in the delivered build.

Layer	Recommended technology	Why it belongs
Frontend	React, TypeScript, Vite; Tailwind CSS; Recharts; TanStack Query	Typed UI, charts, query caching and job polling.
HTTP API	Python, FastAPI, Pydantic v2, Uvicorn	Validated request models and automatic OpenAPI documentation [R1].
Persistence	PostgreSQL; SQLAlchemy 2; Alembic	Relations, transaction boundaries and schema migration.
Money	Python Decimal; SQL Numeric(18,2)	Deterministic arithmetic and explicit rounding [R4, R5].
Background work	Celery with Redis broker; Celery beat scheduler	Independent worker and repeatable scheduled analysis [R3].
AI categorization	scikit-learn TF-IDF and logistic regression	Local classifier, evaluated on held-out data; runs in the worker [R7].
Parsing	Python csv; pdfplumber; parser adapters	Declared CSV format and text-PDF extraction with page lineage.
Verification	pytest, Ruff; TypeScript checks; GitHub Actions	Business tests, static checks and image builds.
Deployment	Dockerfiles; Docker Compose v2; Nginx	One deployment command and same-origin browser API access [R2].

Service	Process or storage	Host exposure
frontend	Nginx serves built UI and proxies /api to api:8000	127.0.0.1:8080
api	Uvicorn app.main:app on port 8000	Internal; browser uses /api
worker	Celery worker runs parsing and analysis tasks	None
scheduler	One Celery beat instance creates scheduled tasks	None
postgres	Financial records and analysis history	None
redis	Task broker with persistence enabled	None
init	One-shot migrations and optional demo seed	None; exits successfully
llm optional	Model server behind an explicit ai profile	Internal only

Baseline needs no vector database, full-text cluster or LLM service. Financial questions can be answered from typed ledger queries and computed insights. Add embeddings only if a later feature needs retrieval from long explanatory documents.

Design note. Select an optional model only after checking memory, license and measured response quality on the target laptop. Do not download model weights during an offline demonstration.

Agent responsibilities and coordination

An agent here is a named task module with a typed contract and recorded execution. Financial calculations are deterministic; categorization includes a local learned classifier. The optional advisor may use an LLM for explanation or tool selection; it must not invent amounts or perform financial actions.

Agent	Input	Output and responsibility
1 Ingestion	Upload and adapter	Normalized rows, review errors and source references.
2 Categorization	Merchant, description, overrides	Category, rule/model source, review flag and version.
3 Spending analysis	Approved ledger and period	Category totals, comparable monthly trends and evidence IDs.
4 Budget planning	Budgets, income assumptions	Utilization, remaining amount and proposed adjustments.
5 Health scoring	Savings, debt and reserve inputs	Component scores or unavailable status, with formula version.
6 Anomaly detection	Current row and historical baseline	Unusual-amount or possible-duplicate candidates.
7 Subscription intelligence	Recurring transactions and feedback	Recurring candidates, expected next date and review status.
8 Cash-flow forecast	Dated balance, expected flows	Daily projected balances, shortfall date and assumptions.
9 Financial advisor	Completed insights and user goals	Ranked actions with calculation and source references.
10 Notifications	Actionable insights and prior alerts	Deduplicated in-app alerts, acknowledgments and reminders.
11 Orchestrator	Job trigger and data version	Dependency order, stage status and atomic snapshot publication.

Execution order

Normalize and categorize first. Spending, anomalies and recurring detection then read the same approved ledger version. Budget and health calculations use the relevant totals and user inputs. Forecasting consumes confirmed obligations. The advisor reads completed outputs; notifications run after publication. Parallelize independent calculations only after the common data version is fixed.

When does the system run again

A successful import, category correction, budget update, balance update or obligation change requests recomputation. The scheduler checks hourly for due obligations and stale data. Two triggers for the same user and data version coalesce into one active run. The UI records when the analysis last completed.

Ingestion and transaction normalization

Supported inputs and processing rules

Accept UTF-8 CSV in the declared schema and text-based PDFs from one explicitly supported statement adapter. Limit demo uploads to 10 MB and 5,000 rows per file. A scan without extractable text is marked OCR_REQUIRED; password-protected or unsupported layouts receive actionable errors. OCR and more bank layouts are good-to-have adapters.

Upload handling checks ownership, extension, content signature and size, writes a generated private filename, computes a SHA-256 digest, and returns a job ID. Never make a source file accessible by a public storage URL. Validation follows the principles in [R6].

```
date,description,amount,direction,currency,reference,balance
2026-03-04,SWIGGY ORDER,450.00,DEBIT,INR,UPI001,25550.00
2026-03-05,SALARY,50000.00,CREDIT,INR,SAL001,75550.00
```

The upload form supplies account_id, account_type and statement period. Each adapter explicitly maps date format, amount signs and debit or credit columns. Do not infer an ambiguous date such as 03/04/2026. Amounts in the normalized ledger are positive decimal strings plus a DEBIT or CREDIT direction.

Pipeline and review

• Parse raw rows into staging; retain original text, page or row location, and parser version.

• Validate dates, currency, positive amount and required fields. Reject NaN, infinity and conflicting debit/credit values. Record an error for each invalid row.

• Reconcile balances when opening, closing and transaction balances are supplied. Flag discrepancies; never fabricate a missing transaction.

• Commit approved rows; send ambiguous rows and uncertain categories to review. Excluded rows are visible and are not counted in published totals.

• Detect an exact repeated file for the same user and account. Use stable transaction references for cross-file identity when available. Otherwise create a possible-overlap review item rather than deleting a legitimate repeated purchase.

Category precedence

User correction > user-approved merchant mapping > configured merchant rule > local trained classifier > Unclassified. Preserve raw descriptions. Train TF-IDF plus logistic regression on labeled synthetic descriptions; evaluate a held-out split with distinct merchant groups. A proposed 0.75 model-score cutoff routes uncertain predictions to review; scores are not calibrated confidence. Corrections update mappings, not model weights.

Credit-card purchases count as spending; repayment from a bank account is a liability settlement. Linked transfers between owned accounts must not appear as income or consumption. Refunds reduce the linked category’s net spending; unresolved links remain flagged.

Data model and financial invariants

Entity	Key fields and relationships
users and accounts	user_id; account_id; account_type; currency; balance and balance_as_of. Every financial entity is scoped to user_id.
uploads and jobs	upload_id; account_id; content_hash; storage_key; parser_version; period; job status and stage. Unique upload hash per user/account.
transactions	transaction_id; account_id; date; amount Numeric(18,2); direction; currency; type; category; reference; review_state.
transaction_sources	transaction_id links to upload_id, page or row, raw value and parse status. A canonical transaction can have multiple source links.
category_overrides	Transaction or merchant mapping, chosen category, user, timestamp and mapping version.
budgets and obligations	Period/category limits; recurring income or payment amount, due rule, account, confirmed status and validity dates.
analysis_runs and insights	run_id; user data_version; config_version; cutoff; state; metrics; insight evidence and assumptions.
alerts and audit_events	Insight deduplication key; unread/acknowledged status; actor; action; correlation ID; timestamp. Optional goals store target, saved amount and due date.

Canonical transaction example

```
{
  "transaction_id": "tx_0042", "account_id": "acc_bank_1",
  "date": "2026-03-04", "amount": "450.00",
  "direction": "DEBIT", "currency": "INR",
  "transaction_type": "EXPENSE", "category": "Food & Dining",
  "category_source": "merchant_rule", "review_state": "APPROVED",
  "source": {"upload_id": "up_001", "row": 2, "page": null}
}
```

Invariants to enforce in code

Money enters as strings, is parsed into Decimal, and is stored as fixed-scale numeric values. Quantize output amounts to two decimals using a documented rounding policy; keep intermediate rates at sufficient precision [R4, R5]. Currency conversion is outside the baseline: reject unsupported currencies instead of summing them.

Income, consumption, transfers, debt settlement and refunds use distinct transaction types. Persist a new data_version after a material edit. A completed analysis references its exact data and configuration versions. Source deletion invalidates dependent outputs and requests a new analysis; it does not leave orphaned advice.

Spending budgets and health indicators

Spending pattern analysis

For each complete calendar month, compute approved category expenses minus linked refunds. Compare equivalent periods only. Monthly change is (current - previous) / previous × 100. If the previous value is zero, show the amount difference and mark the percentage unavailable. Partial-month comparisons require the same day cutoff and an explicit label.

Example: Food spending INR 5,000, then 6,500, then 8,000 produces changes of 30.00% and 23.08%. Report those values instead of describing the whole series as an 18% monthly increase.

Budget management

Budget utilization = net category spending / category limit × 100. Warn at a configurable 90% and mark exceeded at 100% or above. A zero budget gives a zero-limit status when spending exists; do not divide by zero. The user confirms all budget changes.

An optional 50/30/20 suggestion splits INR 50,000 into needs 25,000, wants 15,000 and a savings allocation 10,000. Present it as a configurable starting template. Actual savings and transferred savings are separate measures; do not report a suggested allocation as achieved saving.

Proposed wellness score

This is a transparent prototype heuristic, not a credit score, lending decision or validated financial rating. Display each component and its inputs. Debt obligations and liquid emergency reserves must be explicitly entered or confirmed; ordinary transaction history is insufficient to infer them reliably.

Component	Definition	Share
Savings discipline	S = clamp(net monthly surplus / confirmed net income / target savings rate, 0, 1). Default target rate 0.20.	35%
Budget discipline	B = 1 - clamp(total positive category overspend / total positive budget limits, 0, 1).	25%
Debt burden	D = 1 - clamp(confirmed monthly debt payments / confirmed net income / configured burden ceiling, 0, 1). Default ceiling 0.40.	20%
Reserve coverage	E = clamp(liquid reserve / average essential monthly spending / configured target months, 0, 1). Default target 3 months.	20%

Score = round(100 × (0.35S + 0.25B + 0.20D + 0.20E)). Publish the total only when all four inputs are available and denominators are valid; otherwise show the available components and “incomplete”. clamp(x,0,1) bounds a value to that interval. Targets and thresholds are configurable design defaults, not universal advice.

Anomalies and subscription intelligence

Unusual spending detector

For a candidate expense, use only approved earlier expenses for the same category in the previous 90 days. Require at least ten observations. Let m be the median and MAD the median absolute deviation from m. Flag a review candidate if amount > max(3m, m + 6MAD, INR 2,000). These are demo defaults to tune and validate, not a trained fraud model.

Example: earlier amounts centered at INR 500 with MAD INR 100 give a threshold of INR 2,000. An INR 15,000 debit is unusual by this rule. Explain the baseline and threshold; ask the user whether it was expected. With insufficient history, return INSUFFICIENT_HISTORY rather than a claim of normality.

Possible duplicate charges

Different references with the same normalized merchant and amount within one day create a possible-duplicate candidate. Same-reference overlapping statement rows are an ingestion identity issue. Both require explicit handling: a valid repeat purchase remains in spending; a confirmed duplicate remains a posted charge until a refund arrives. Never silently erase a suspected charge.

Recurring payments and subscriptions

Group expenses by normalized merchant. A monthly recurring candidate requires at least three charges, successive gaps between 25 and 35 days, and amounts within 10% of the median. This identifies a recurring pattern, not proof of a subscription. Ask the user to confirm whether it is a subscription, bill or installment.

The next expected date is the last date plus the median observed gap. Once the user confirms a due rule, use that rule for forecasting. Keep cancellation state and expected end date; stop future projections only after confirmation. Variable bills may use a user-approved estimate.

Evidence available	Allowed finding
Regular posted charges only	Recurring payment detected; ask whether the service is still needed.
User marks a service unused	Potential cancellation saving, based on future avoided payments.
No charge in a recent month	Pattern may have ended or data may be incomplete; ask for confirmation.
Usage feed with explicit consent	Optional usage-based review; do not infer usage from absence of bank activity.

If a confirmed monthly subscription costs INR 599, cancellation could avoid INR 599 per future month or INR 7,188 over twelve such months. Label the horizon and assumptions. Do not claim savings already realized or add this estimate to another plan that includes the same charge.

Cash flow forecasts and goal planning

Baseline forecast algorithm

Forecast 30 days from a user-confirmed liquid balance and balance_as_of date. For day d: projected_balance(d) = opening_balance + cumulative expected inflows through d - cumulative expected cash outflows through d. Use a conservative within-day ordering of outflows before inflows to expose temporary shortages.

Projected cash outflows include confirmed bills, EMIs, subscriptions and a daily allowance for variable cash spending. Estimate the allowance from up to three complete months, excluding recurring items and transfers already modeled. Actual posted flows before the forecast cutoff must not be replayed as future flows.

For multiple cash accounts, combine balances only when they share a declared cutoff. Model a bank-to-card repayment as a cash outflow without treating it as new consumption. Historical card purchases affect spending indicators; future card settlement affects liquid cash. Unsupported account reconciliation produces an incomplete forecast.

Illustrative 30-day input	Amount or result
Opening liquid balance on April 1	INR 32,000
Confirmed salary on April 5	+ INR 50,000
Rent, EMI and utilities on specified dates	- INR 31,000
Confirmed subscriptions	- INR 1,500
Variable cash spending allowance	INR 600 × 30 days = - INR 18,000
Projected closing balance	INR 31,500
Scenario without April salary	INR -18,500 closing balance; compute first negative day from the dated events

Show a day-by-day curve, the first day below the user’s safety buffer, and the first negative day. These are scenario estimates, not prediction intervals. Flag income dates inferred from history as unconfirmed. If no dated opening balance exists, provide net-flow estimates and request the missing input.

Good-to-have goals and scenarios

Required monthly contribution = max(0, target amount - amount already saved) / whole months remaining, rounded up to the next paisa. Example: INR 2,04,000 remaining over 24 months requires INR 8,500 per month. A past due date is invalid; a current-month deadline needs an immediate contribution.

A scenario clones the current forecast and changes explicit inputs such as salary loss, new rent or a supplied loan payment. Return baseline versus scenario balance, shortfall date and assumptions. If calculating an EMI, require principal, monthly interest rate and installment count; handle zero interest as principal / count. Do not assume an interest rate or promised return.

Advisor orchestration and evidence

What makes the workflow agentic

The orchestrator responds to a trigger, checks which inputs are available, runs the relevant tools, records incomplete stages and publishes a prioritized plan. It revisits the plan after new data or user feedback. This closes the analysis-to-action loop without requiring autonomous bank transactions.

Baseline advice generation

Generate recommendations from typed insight templates. Rank confirmed upcoming shortfalls first, then exceeded budgets, unusual-activity review and optional saving opportunities. Each recommendation contains an observation, evidence, calculation, proposed action, uncertainty and a next review date. Use the user’s declared goals to choose relevant actions.

```
{
  "insight_id": "ins_017", "run_id": "run_008",
  "type": "BUDGET_RISK", "severity": "MEDIUM",
  "observation": "Food budget is 90% used",
  "metric": {"spent": "7200.00", "limit": "8000.00",
             "utilization_pct": "90.00", "currency": "INR"},
  "evidence": {"transaction_ids": ["tx_01", "tx_02"],
               "period": "2026-03", "rule": "budget-v1"},
  "action": "Review planned dining expenses; INR 800 remains",
  "assumptions": ["Only approved posted expenses are included"]
}
```

Optional conversational advisor

Expose allowlisted read tools: get_month_summary, get_budget_status, get_forecast, get_insight_evidence and simulate_scenario. The server resolves the user from authentication, validates tool arguments and caps calls. The model can explain computed results but cannot create transactions, cancel a service or alter a budget.

Cite insight IDs and source rows or pages that open in the UI. If a question needs missing debt data or a future income confirmation, request that input. Treat statement descriptions as untrusted data, including any text that looks like an instruction. A timeout or invalid model response falls back to the deterministic insight cards.

Notifications and action tracking

Create in-app alerts from published insights. Deduplicate using user + type + entity + period or due date; update severity instead of creating identical alerts on every run. Track unread, acknowledged and resolved states. Acknowledgment records review; it does not imply the financial risk disappeared. Record completed actions only after user confirmation or new evidence.

Frontend screens and user journey

Screen	Contents and required interaction
Dashboard	Income, net spending, surplus, budget utilization, wellness components, upcoming bills, latest analysis time and data coverage. All cards link to details.
Upload and review	Choose account and statement period; upload; poll stage and row counts; review errors; confirm import. Show supported file layouts before upload.
Transactions	Paginated ledger with date, amount, type, category, account and source. Filters by month/category/account; editable category and transfer linkage.
Budgets	Monthly category limits, spent/remaining/utilization and suggested allocation. Budget changes require confirmation and trigger re-analysis.
Insights and alerts	Severity, reason, threshold, evidence list and review action. User marks an expense expected, reviews a possible duplicate, or confirms a recurring item.
Forecast and goals	Daily balance curve, bills list, opening balance/date, scenario assumptions and insufficient-data indicators. Optional target contribution planner.
Advisor optional	Questions answered from approved tools with clickable references. Explicit unavailable state if model service is disabled.
Settings and system	Account balances, obligations, goals, privacy controls, agent run history and measured service readiness.

End-to-end interaction

1. Sign in to a synthetic demo user. 2. Upload the three-month fixture. 3. See the job complete and review uncertain rows. 4. Open the dashboard. 5. Inspect a budget alert and its source. 6. Correct a category. 7. Enter or confirm the forecast balance and bills. 8. Compare a scenario. 9. Acknowledge an alert and show the audit record.

Network and display behavior

Use TanStack Query to fetch typed responses and poll active jobs every two seconds. Poll analysis and system status less often when idle. Invalidate relevant queries after an edit; keep displaying the previous completed snapshot with a “recomputing” label until the new run succeeds. Show loading, empty, partial, stale and failed states explicitly.

Service indicators come from backend checks and worker or scheduler heartbeats, not hard-coded ticks. State whether the result is healthy, degraded, unavailable or unknown. The latest uploaded statement date must remain visible so the user can distinguish fresh processing from fresh financial data.

REST API and OpenAPI contract

All domain routes use /api/v1 externally; the reverse proxy strips /api so the backend sees /v1. Require authentication and ownership checks on financial routes. Serve Swagger at /api/docs and the schema at /api/openapi.json through the same proxy [R1].

Method and backend route	Purpose and principal response
POST /v1/auth/login	Validate seeded account credentials; issue an opaque HttpOnly session cookie. GET /v1/auth/me and POST /v1/auth/logout complete the session flow.
POST /v1/uploads	Multipart file plus account_id and period; require Idempotency-Key; return 202 with upload_id and job_id.
GET /v1/jobs/{id}	State, stage, counts, review errors and safe error code; only the owning user can read it.
POST /v1/uploads/{id}/confirm	Commit review decisions; return analysis job_id. GET /v1/uploads/{id}/source serves owned source evidence.
GET /v1/transactions	Period/account/category filters, page and page_size; return rows and total. Maximum page_size 100.
PATCH /v1/transactions/{id}	Category, review decision or transfer linkage; version precondition; return updated record and analysis job_id.
GET and PUT /v1/budgets/{month}	Read or set category limits. PUT replaces the submitted month configuration; reject invalid or negative limits.
GET /v1/dashboard	Completed summary with period, data_version, computed_at and coverage.
GET /v1/insights and /{id}	List filtered insights or inspect evidence. GET /v1/alerts returns notification state.
PATCH /v1/alerts/{id}	Set acknowledged or resolved state with a reason. Preserve audit history.
GET and PUT /v1/accounts/{id}/balance	Read or confirm dated balance. GET/POST/PATCH /v1/obligations manage expected flows.
POST /v1/analysis/runs	Explicit rerun; return 202 and job_id. GET /v1/forecast returns latest completed projection.
GET /v1/exports/transactions.csv	Download filtered ledger, with spreadsheet formula-injection protection.
Optional /v1/goals, /scenarios, /advisor	Goal CRUD; POST scenario comparison; POST grounded question and answer.
GET /health/live and /health/ready	Liveness and DB/queue readiness. GET /v1/system/status adds worker/scheduler heartbeat status.

Design note. Generate and commit openapi.json after contract freeze. Each route must include request/response examples, documented auth and expected errors. Paths above are proposed contracts, not implemented endpoints.

Configuration jobs and error contracts

Data-driven rules

Store rules in versioned, schema-validated JSON. Use typed thresholds and allowlisted operators; do not execute arbitrary code or eval expressions. Validate a complete configuration before replacing the active version. Existing insights retain the rule version that produced them.

```
{
  "version": "finance-v1", "currency": "INR",
  "budget": {"warning_pct": "90.00"},
  "anomaly": {"history_days": 90, "minimum_samples": 10,
              "median_multiplier": "3", "mad_multiplier": "6",
              "minimum_amount": "2000.00"},
  "recurring": {"minimum_charges": 3, "gap_min_days": 25,
                "gap_max_days": 35, "amount_tolerance": "0.10"},
  "forecast": {"horizon_days": 30},
  "monitoring": {"schedule_minutes": 60, "stale_after_days": 7}
}
```

Job states and durability

QUEUED → RUNNING → NEEDS_REVIEW or SUCCEEDED. Validation failure becomes FAILED with a safe error code. Confirmation creates a commit-and-analysis job. Use a PostgreSQL job record and a dispatch/outbox marker so a queue outage cannot lose accepted work; a scheduler sweep republishes undispatched jobs.

Queue delivery may repeat. Enforce database uniqueness for imports and alert keys, claim a run with a per-user database lock or leased job, and persist completed stages. Use bounded retries with backoff for transient errors. Do not retry invalid inputs. Celery retries do not make tasks idempotent automatically [R3].

Before publication, compare the run’s input data_version with the current user version. If it changed mid-run, mark the run superseded and enqueue the latest version. Publish metrics and insights in one database transaction; preserve the previous completed snapshot if the new run fails.

Error response

```
{"error": {"code": "UNSUPPORTED_STATEMENT_LAYOUT",
           "message": "Use the provided CSV template or supported PDF",
           "request_id": "req_019", "details": {"upload_id": "up_07"}}}
```

Use 401 for no valid session, 404 for an inaccessible resource, 409 for conflicting edits or key reuse with different content, 413 for upload size, 422 for validation and 503 for dependency failure. Store correlation IDs in operational logs without raw statement content or credentials.

Docker operation and readiness

Single-command startup requirement

The repository must include compose.yaml, application and frontend Dockerfiles, lockfiles, .env.example, bootstrap code and health checks. A judge runs the following in the repository root using Windows PowerShell. Docker Desktop must already be running with Linux containers.

```
Copy-Item .env.example .env
# Edit .env in the IDE and set the documented local secrets.
docker compose config --quiet
docker compose up --build -d
docker compose ps -a
```

Open http://localhost:8080 for the app and http://localhost:8080/api/docs for Swagger. The startup command builds missing images, starts PostgreSQL and Redis, runs init, then starts API, worker, scheduler and frontend. init should appear Exited (0), while runtime services remain running.

What the developer must supply

Initialize schema through Alembic in init only; then seed synthetic users and fixtures only when DEMO_SEED=true. Repeated initialization must not duplicate fixtures. Install health-check dependencies in the images. Ship Swagger assets locally if the demo must work without internet. Copy the rules directory into the application image.

Use depends_on with service_healthy for stateful dependencies and service_completed_successfully for init [R2]. API readiness checks database and broker access; worker and scheduler health use recent heartbeats stored in PostgreSQL. A container merely running is not proof that analysis is working.

Logs stop and verification

```
docker compose logs --tail 100 api worker scheduler init
docker compose exec api python -m app.smoke_test
docker compose down
# Restart later; named data volumes remain.
docker compose up -d
```

The smoke test must authenticate as a fixture user, upload a fixture, wait for completion, verify a known insight and exit nonzero on failure. A routine stop must preserve named volumes; do not include volume deletion in normal shutdown instructions.

Proposed performance targets

On a documented 4-core CPU / 8 GB RAM machine, with images already built and no optional model: target ready UI and services within 60 seconds, 1,000 CSV rows analyzed within 30 seconds, and typical dashboard reads under 500 ms at the 95th percentile. Measure and report actual results; cold builds, image pulls and model downloads are excluded from the startup target.

Compose reference template part one

The following two pages form one proposed compose.yaml. It becomes runnable after the referenced code and Dockerfiles are implemented. This document does not include a built repository or verified images. Set POSTGRES_USER, POSTGRES_PASSWORD and SESSION_SECRET in .env; DEMO_PASSWORD is required only for demo seeding.

```
x-app: &app
  build: ./backend
  env_file: .env
  volumes:
    - uploads:/data/uploads
  restart: unless-stopped

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: finance
      POSTGRES_USER: ${POSTGRES_USER:?Set POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?Set POSTGRES_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: [CMD-SHELL, 'pg_isready -U "$${POSTGRES_USER}" -d finance']
      interval: 5s
      timeout: 3s
      retries: 20
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    command: [redis-server, --appendonly, 'yes']
    volumes:
      - redisdata:/data
    healthcheck:
      test: [CMD, redis-cli, ping]
      interval: 5s
      timeout: 3s
      retries: 20
    restart: unless-stopped

  init:
    <<: *app
    command: [python, -m, app.bootstrap]
    depends_on:
      postgres:
        condition: service_healthy
    restart: 'no'

  api:
    <<: *app
    command: [uvicorn, app.main:app, --host, 0.0.0.0, --port, '8000']
    depends_on:
      init:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    healthcheck:
      test: [CMD, python, -m, app.healthcheck, api]
      interval: 5s
      timeout: 4s
      retries: 20
```

Design note. The major image tags above are baseline examples. Replace them with tested fixed patch tags or digests before delivery. PostgreSQL and Redis are reachable only on the Compose network, without published host ports.

Compose reference template part two

```
  worker:
    <<: *app
    command: [celery, -A, app.tasks.celery_app, worker, --loglevel=info]
    depends_on:
      init:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    healthcheck:
      test: [CMD, python, -m, app.healthcheck, worker]
      interval: 10s
      timeout: 4s
      start_period: 30s
      retries: 6

  scheduler:
    <<: *app
    command: [celery, -A, app.tasks.celery_app, beat, --loglevel=info]
    depends_on:
      init:
        condition: service_completed_successfully
      redis:
        condition: service_healthy
    healthcheck:
      test: [CMD, python, -m, app.healthcheck, scheduler]
      interval: 10s
      timeout: 4s
      start_period: 30s
      retries: 6

  frontend:
    build: ./frontend
    ports:
      - '127.0.0.1:8080:80'
    depends_on:
      api:
        condition: service_healthy
    healthcheck:
      test: [CMD, wget, -qO-, http://127.0.0.1/healthz]
      interval: 10s
      timeout: 3s
      retries: 6
    restart: unless-stopped

volumes:
  pgdata:
  redisdata:
  uploads:
```

Application configuration and browser routing

The application Settings class must build its database URL from POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB=finance and POSTGRES_HOST=postgres. Set REDIS_URL=redis://redis:6379/0, UPLOAD_DIR=/data/uploads and RULES_PATH=/app/rules/finance_rules.json. SESSION_SECRET has no default. Commit .env.example with explanatory placeholders; exclude .env from Git.

Nginx must map /api/ to http://api:8000/ with the trailing slash, return 200 at /healthz, and serve the SPA fallback. The browser calls /api/v1/... rather than the internal hostname api. Configure FastAPI root_path=/api so Swagger loads the correct schema URL through the proxy.

Only one scheduler instance runs in this baseline. Its periodic task writes a heartbeat and dispatches due jobs. Worker heartbeat must update while idle as well as busy. Add an optional model through a separate ai profile; it must not become a dependency for baseline startup.

Repository structure and coding roadmap

Agree on the file boundaries before building. The names below match the Compose entrypoints and API contracts; they are a proposed project layout.

Path	Responsibility
compose.yaml and .env.example	Runtime graph, environment documentation and named volumes.
backend/Dockerfile and dependency lock	Repeatable Python image and locked package installation.
backend/app/main.py and api/	FastAPI app, auth, uploads, ledger, budgets, insights, forecast and system routers. Keep routers thin.
backend/app/schemas/ and models/	Pydantic request/response contracts and SQLAlchemy entities.
backend/app/services/	Ownership checks, use-case transactions and repositories. Shared business functions used by API and jobs.
backend/app/agents/	ingestion.py, categorization.py, spending.py, budget.py, health.py, anomaly.py, recurring.py, forecast.py, advisor.py, notifications.py.
backend/app/orchestrator.py	Stage dependencies, input version checks, run publication and deduplication.
backend/app/tasks.py	celery_app, import/analyze tasks, scheduled dispatch and heartbeat tasks.
backend/app/bootstrap.py and healthcheck.py	Migrations/seed entrypoint; API and process health checks.
backend/app/smoke_test.py	Deterministic end-to-end probe with a timeout and failure exit code.
backend/app/parsers/, rules/ and ml/	CSV/PDF adapters; validated rules; classifier training script and versioned model artifact.
backend/alembic/ and tests/	Schema migration files and unit/integration/contract tests.
frontend/src/pages/ and components/	Screens, shared ledger table, evidence panel and chart components.
frontend/src/api/ and hooks/	Typed HTTP adapter, auth handling, query keys and job polling.
frontend/Dockerfile and nginx.conf	Build UI; same-origin API proxy; SPA routing and /healthz.
data/fixtures/ and docs/	.csv/.pdf synthetic files, golden outputs, API_CONTRACT.md, architecture and README.
.github/workflows/ci.yml	Static checks, tests and container build verification.

Implementation discipline

Start with one uploaded CSV reaching one computed insight, then extend the same flow. Keep calculations as pure functions receiving explicit data, configuration and dates. Put persistence and scheduling outside those functions. No agent should depend on a UI component or on another agent’s hidden state.

Detailed task breakdown

Basic tasks Must have

• **1 Freeze contracts.** Define the canonical ledger, review states, money representation, account types, insight evidence and endpoint schemas. Deliver docs/API_CONTRACT.md plus example payloads.

• **2 Establish deployment.** Create locked manifests, Dockerfiles, database migrations and init. Verify an empty dashboard and health endpoints through the frontend proxy.

• **3 Implement the data domain.** Build CSV staging, one PDF adapter, validation, lineage and import identity. Deliver a reviewable ledger and a repeat-import test.

• **4 Implement categorization and spending.** Add merchant rules, train/evaluate the local classifier, then user corrections, transfer/refund treatment and month totals. Verify against golden outputs.

• **5 Deliver budget monitoring.** Save category limits, compute utilization and create evidence-backed alerts. Show a correction triggering a new run.

Intermediate tasks Must have

• **6 Add intelligence.** Implement anomaly candidates, recurring detection, confirmation of obligations, 30-day forecasting and incomplete wellness components.

• **7 Complete orchestration.** Wire worker, scheduler, persisted job states, retry bounds, heartbeat checks, atomic publication and alert deduplication.

• **8 Complete the UI.** Add review actions, evidence links, forecast assumptions, filtering, pagination, protected export and loading/error states.

• **9 Prove reproducibility.** Add OpenAPI examples, automated tests, CI image builds, seed idempotency and a clean-clone runbook. Test restart persistence and two-user isolation.

Advanced tasks Good to have

• Add goal-based planning and a baseline-versus-scenario financial twin using the same deterministic forecast functions.

• Add an optional tool-using conversational advisor with source references, model timeouts and deterministic fallback.

• Add OCR, new statement adapters, usage-based subscription evidence, household permissions or optional email notification simulation.

Delivery milestones and stop conditions

Milestone A: upload-to-ledger works. B: ledger-to-budget-alert works. C: recurring obligations-to-forecast works. D: scheduler, tests and Compose are reliable. E: optional features. Timebox only after the actual hackathon duration is known. If time runs short, stop at a complete tested baseline; do not replace real calculations with decorative alerts.

Inputs datasets and golden examples

No transaction dataset was supplied with these two briefs. Create synthetic fixtures as part of implementation. Suggested scale: three complete months, two users, two cash accounts and one credit card, with roughly 300-1,000 rows. Keep IDs stable. Tests use a fixed clock; if the demo uses April 1, 2026, visibly label it as simulated time.

Fixture or file	Purpose and required content
data/fixtures/statements/*.csv	Declared normalized upload schema plus date-format metadata; salary, expenses, transfers, refunds and repayment examples.
data/fixtures/statements/*.pdf	One supported text-PDF layout and an unsupported or image-only fixture. Include page-linked source rows.
accounts.json and profile.json	Account types, INR balances with a shared cutoff, confirmed income, debt payment and liquid reserve inputs.
budgets.json and obligations.json	Monthly category limits; confirmed due dates, payment amounts and applicability periods.
rules/finance_rules.json	Validated defaults matching page 15 and a changed version for recomputation tests.
labeled_transactions.csv and expected/*.json	Labeled classifier training/holdout records; golden totals, category results, alerts and forecast balances.
queries.json optional	Questions with expected tool calls and evidence IDs, including missing-data and unsafe-action requests.

Golden cases to encode

• **Budget:** food spend INR 7,200 / limit INR 8,000 = 90.00% and INR 800 remaining. One budget-risk alert after repeated runs.

• **Trend:** INR 5,000 → 6,500 → 8,000 across full months yields 30.00% then 23.08%. Zero and partial-month baselines are separate fixtures.

• **Recurring:** INR 599 charges on January 5, February 5 and March 5 satisfy the 25-35 day gaps; usage remains unknown until the user provides feedback.

• **Anomaly:** ten or more earlier category observations with median INR 500 and MAD INR 100; INR 15,000 creates a review candidate under page 10.

• **Data identity:** exact reupload contributes no extra spending; two same-amount purchases with distinct references both remain posted.

• **Cash flow:** the page 11 event schedule closes at INR 31,500; removing the salary closes at INR -18,500. Store dated bills to assert the first negative day.

• **Classification:** owned-account transfer and card repayment are not consumption; a linked refund reduces the relevant category. These cases must also reconcile cash balances.

Design note. Example amounts from the original brief are illustrative claims, not supplied measurements or ground truth. Golden outputs must be calculated from the fixtures actually created.

Testing CI and engineering quality

Use meaningful tests with expected outcomes independent of implementation. The proposed minimum is twelve; the following cases define useful coverage. Keep business tests fast and use database-backed integration tests for identity and ownership.

Case	Required assertion
T01 Valid import	CSV and supported PDF normalize expected dates, amounts and source references.
T02 Invalid input	Oversize, ambiguous date, bad amount and unsupported layout yield visible safe errors.
T03 Idempotency	Repeat upload or job delivery creates no duplicate ledger entries, seed rows or alerts.
T04 Category correction	Override changes analysis; held-out classifier metrics and review coverage are reported; ownership is enforced.
T05 Financial types	Transfer, repayment and refund handling produces correct spending and cash movements.
T06 Budget edge cases	90% warns; exceeded limit flags; zero limit and zero spending do not divide by zero.
T07 Trend coverage	Complete-month math matches expected values; partial and zero baselines are labeled.
T08 Anomaly history	Only prior transactions enter the baseline; insufficient history yields unavailable status.
T09 Recurring review	Known gaps produce a candidate; no finding claims unused service without evidence.
T10 Forecast arithmetic	Dated events match every daily balance; no double-counting recurring charges or repayments.
T11 Missing inputs	Missing balance, debt or reserve produces incomplete outputs, not fabricated values.
T12 Ownership and auth	Unauthenticated reads fail; guessed IDs cannot expose another user’s files or insights.
T13 Crash and stale run	Worker retry recovers; input change supersedes old analysis; previous snapshot remains usable.
T14 End to end	Clean init, upload, review, alert and forecast work via the frontend API path; restart preserves state.

CI sequence

On each push: install locked dependencies; run Ruff and TypeScript checks; execute unit tests; start a PostgreSQL/Redis test environment for integration tests; build frontend; build Docker images; validate Compose configuration with dummy CI secrets. Add the end-to-end smoke job once the baseline is stable. CI must not import private statements or expose secrets.

Engineering handover checklist

Deliver meaningful commits, typed contracts, precise money calculations, safe logs, locked dependencies, migration history, deterministic fixtures and a clean-clone README.

Guardrails and failure handling

Privacy ownership and human control

Use synthetic data for judging. For this local baseline, create two fixture users with hashed passwords and server-side sessions. Session cookies are HttpOnly and SameSite; apply CSRF protection to mutations. Use Secure cookies and TLS for a real deployment. Set expiry, revoke on logout and never log credentials or full session tokens.

Authorize every financial query and source download by the authenticated user. Mask account identifiers, keep uploads outside the web root, and implement data deletion across sources, derived insights and alerts. A shared local volume is not an encryption control; a real deployment requires protected storage, key management and a retention policy.

The assistant may recommend or simulate. It does not send payments, access bank credentials, trade investments, apply for credit or cancel subscriptions. Budgets, classifications and obligation changes require an explicit user action. Alerts indicate review needs, not confirmed fraud.

Input and model guardrails

Apply size and type limits, generated filenames and resource limits during parsing [R6]. Protect CSV exports from formula execution. Keep statement text out of model prompts unless explicitly needed and permitted; minimal structured context is preferred. Tool authorization belongs on the server. Model output is validated data and is never executable code.

Failure or uncertainty	Required response
Unsupported or scanned PDF	Explain supported input and offer the CSV template; mark OCR as optional.
Missing rows or reconciliation gap	Publish coverage and review warnings; do not call the statement fully reconciled.
Missing current balance or stale history	Request the balance/cutoff or show an incomplete forecast and source freshness.
Redis or worker unavailable	Keep accepted job in PostgreSQL; show queued/degraded status; retry dispatch safely.
New analysis fails	Keep last completed snapshot with its timestamp and a visible failure message.
Unknown recurring charge usage	Ask for confirmation; never label unused automatically.
LLM down or unsupported question	Use computed insight cards or ask for missing information.
Scheduler repeats an insight	Update the existing alert using its deduplication key.

Guardrails above are prototype requirements and deployment considerations, not a claim of legal, banking or security certification. Before accepting real-user data, the owner must define applicable consent, access, retention and operational controls.

Demonstration handover and references

Suggested demonstration sequence

Show the data coverage first. Upload the synthetic three-month statement, watch real job stages, then open one source row. Show a 90% budget alert, a recurring payment and an unusual transaction with their evidence. Change one category and show recomputation. Open the forecast, remove the assumed salary in a scenario and explain the resulting shortfall. Finish by showing one-command startup, service status, OpenAPI and passing tests. Do not claim financial health improved merely because a recommendation was displayed.

Final deliverables

A repository with README, architecture notes, API contract, lockfiles, migrations, rules, fixtures and golden outputs; all application source and Dockerfiles; compose.yaml and .env.example; CI configuration; measured performance results; a concise demo script; and a known-limitations list. These are implementation deliverables to build from this specification, not included software in this PDF.

Source documents and attribution

S1: PS-05_ Personal Finance Agent.docx, supplied by the user. Product intent, the ten agent responsibilities, master orchestration and suggested advanced features originate here.

S2: 2026 Use Case - Reg IRIS vfinal8615346.pdf, supplied by the user, six pages. Used for structure and depth: domains, architecture, done criteria, stack, tasks, inputs, guardrails and engineering checklist. This specification is not issued or endorsed by BNP Paribas.

Official technical references

R1 FastAPI features and background work
https://fastapi.tiangolo.com/features/
https://fastapi.tiangolo.com/tutorial/background-tasks/

R2 Docker Compose startup dependencies
https://docs.docker.com/compose/how-tos/startup-order/

R3 Celery task retries and idempotency
https://docs.celeryq.dev/en/stable/userguide/tasks.html

R4 Python decimal arithmetic
https://docs.python.org/3/library/decimal.html

R5 SQLAlchemy numeric types
https://docs.sqlalchemy.org/en/20/core/type_basics.html

R6 OWASP file upload guidance
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

R7 scikit-learn text features and classifiers
https://scikit-learn.org/stable/modules/feature_extraction.html

Decisions to confirm during implementation

Confirm hackathon duration, supported statement layout, demonstration hardware, whether independently deployed microservices are required, and whether an optional LLM is permitted. Ship the evaluated local categorization model in the baseline image; retrain reproducibly before building it. Until then, use the explicit baseline in this document. Official technical references were checked on 12 October 2026; application formulas and acceptance targets are proposed design choices.